

# BUILD A ROSTER THAT CANNOT STORE LAST NAMES

Twelve tasks. At the end you have a class roster that lives in a real database, opens on any computer you own, and **physically cannot hold a student's surname** — because the database itself refuses one.

**12**

tasks, each ending in something that works

**2**

working files included

**\$0**

to run it, at classroom size

**This is the step up from a single file, and it is a real step.**

Everything in this line so far saves to one computer. This one has a database behind it, so the same roster is there on your laptop, on the classroom desktop, and on the machine in the back room. That is the thing a one-file tool cannot do, and it is why a tool that cannot hold a roster cannot run a course.

**You do not need a coding background — you do need ninety minutes**

Nothing is installed. There is no build step and no npm. You paste one file into a web page once, and everything after that happens in a browser.

## Inside

Twelve tasks with the exact prompts written out in full · the six failures you will actually hit and the sentence that fixes each · the finished roster app and its database file, both included · an honest page about the key in your file and who can read your data · and the page that tells you when you have outgrown a one-file tool.

## BEFORE YOU BEGIN

### The twelve tasks at a glance

Each task ends somewhere you can see. If a task does not reach its end point, do not move on — everything after it assumes that one worked.

| #  | Task                                                  | Ends with                          |
|----|-------------------------------------------------------|------------------------------------|
| 1  | Decide what this tool will never know                 | One sentence, written down         |
| 2  | Make the database                                     | A project URL and a key            |
| 3  | The two tables                                        | Tables that exist                  |
| 4  | <b>The line that makes the wrong thing impossible</b> | The database refusing a surname    |
| 5  | A page that talks to it                               | Your class list on screen          |
| 6  | Shorten in the browser, before it sends               | A live preview as you type         |
| 7  | Paste a whole roster at once                          | A class added in one go            |
| 8  | Two students, same short name                         | Both of them, kept apart           |
| 9  | Prove it to yourself                                  | A refusal you triggered on purpose |
| 10 | Print it                                              | A clipboard-ready list             |
| 11 | The de-identified copy                                | S1, S2, S3 on your clipboard       |
| 12 | What happens in June                                  | A deletion you can actually do     |

#### Stop anywhere after task 6

A roster that lists your class and refuses surnames is already useful. Tasks 7 to 12 make it pleasant. Nobody has to finish a unit to get value out of it.

WHY THIS ONE IS DIFFERENT

When one file stops being enough

Every build in this line so far produces a single page that saves to the computer it is open on. That is the right shape for most classroom tools, and it is the wrong shape for this one.

|                           | A single file                  | What you are building here  |
|---------------------------|--------------------------------|-----------------------------|
| Where data lives          | In one browser, on one machine | In a database you own       |
| Second computer           | Starts empty                   | Same roster, already there  |
| If the browser is cleared | Everything is gone             | Nothing is gone             |
| Setup                     | Open the file                  | About ninety minutes, once  |
| Cost                      | Nothing                        | Nothing, at classroom size  |
| Honest risk               | You lose your data             | Somebody else could read it |

**The sentence a one-file tool cannot say:** a tool that cannot hold a roster cannot run a course. The moment you want the same list of students in front of two different machines — or want it to still exist after a district IT refresh wipes browser data — you have outgrown one file, and no amount of cleverness inside that file fixes it.

And the honest cost, on page one rather than buried

Putting data on a server means somebody other than you could reach it. That is a real trade and this pack does not pretend otherwise — it is the whole reason the build is shaped the way it is. There is a full page on exactly who can read your roster and what they would see.

## Three layers, and none of them trusts the others

Most software protects student data by being careful. Careful software has a bad day eventually. This one is built so the bad day is boring.

### 1 The browser shortens the name

You paste Anthony Spinozzi. Before anything is sent anywhere, the page turns it into Anthony S. The surname never leaves the computer you typed it on.

### 2 The database refuses the long form

One line of the table definition says what a name is allowed to look like. Send Anthony Spinozzi anyway — by accident, by a bug, by pasting into the wrong box — and the database rejects it. Not a warning. A refusal.

### 3 There is nowhere to put one

No column in the whole database is for a last name. Even if somebody wanted to store one, there is no shelf to put it on.

#### Each layer is independent, and that is the entire point.

Layer 1 is code you wrote, so it can have a bug. Layer 2 is a rule the database enforces whatever your code does. Layer 3 is the shape of the thing itself. For a surname to end up stored, all three have to fail at once — and they fail in different ways for different reasons.

#### Why this is worth doing properly once

You can reuse this pattern in everything you build afterwards. Decide what the tool must never know, then make it structurally unable to know it. That habit is worth more than any single tool in this series.

## Before task 1

### A free Supabase account

Supabase gives you a Postgres database with a free tier that is far larger than a roster will ever need. No card. You will make one project and never touch the billing page. **Sign up with your own address, not a school one** — this is your tool, and a district account can be closed out from under you.

### An AI chat you are signed in to

Any of them. You will paste prompts and read what comes back. You are the one at the keyboard for all of it.

### A browser, and somewhere to keep one file

The roster page is a single HTML file that lives in your own Drive or Documents folder. **It is never published to the web**, and there is a page later explaining exactly why that matters.

### What you do not need

Node, npm, a terminal, a deploy pipeline, a domain, a credit card, or permission from anybody — you are building a tool for your own classroom out of your own class list, the same as a spreadsheet.

**One honest warning about time.** Task 2 is account setup and it is the duller ninety seconds in the pack. Tasks 3 and 4 are where it gets interesting. If you only have twenty minutes today, do tasks 1 to 4 — that is the part worth the evening.

## TASK 1

### Decide what this tool will never know

Before a single prompt. This takes four minutes and it is the only task that cannot be done by the AI.

#### Write down one sentence

"This roster knows *<what>* and never knows *<what else>*."

Mine is: **it knows a first name and a last initial, and it never knows a surname, a student ID, a grade, or a birthday.**

#### Why the sentence comes first

If you start prompting without it, the AI will helpfully offer you an email field, a parent contact field and a notes box, because that is what rosters usually have. Every one of those is a thing you would then have to protect. The cheapest student data to keep safe is the kind you never collected.

#### The test for whether a field belongs

For each thing you are tempted to store, ask: **what does this tool do differently because it knows that?**

A roster does nothing differently for knowing a birthday. So it does not get one. If you cannot answer the question, delete the field.

**This is also the answer you give an administrator.** Not "it is secure" — that is a claim you would have to defend. "It cannot hold that" is a fact about the thing, and you can show them the line that makes it true.

## TASK 2

# Make the database

Ends with two strings written down. That is the whole task.

1

### Sign up and make a project

supabase.com, new project, any name. Choose a region near you. It takes a minute or two to finish building — that wait is normal.

2

### Set a database password and keep it

Put it wherever you keep passwords. You will almost never need it, and "almost never" is exactly when people lose things.

3

### Copy two values

Settings, then API. You want the **Project URL** (it looks like `https://abcdefgh.supabase.co`) and the **anon public** key, which is long. Do not copy the one labeled service role.

### Do not copy the service role key. Ever.

Same page, different row. The anon key is meant to sit in a page. The service role key ignores every rule you are about to write, including the one that refuses surnames. You will not need it and nothing in this pack uses it.

### Task 2 ends when

You have a project URL and an anon key in a file you can find again.

### TASK 3

## The two tables

Two tables is the whole database: classes, and the students in them.

#### PROMPT 1 · THE TABLES

Write me a Postgres schema for a class roster, as SQL I can paste into the Supabase SQL editor. Two tables. **classes:** id, name, an optional period, an archived flag, created\_at. **students:** id, class\_id referencing classes with on delete cascade, display\_name, a sort\_order integer, created\_at. Use uuid primary keys with gen\_random\_uuid(). Make display\_name unique within a class. **Do not add any other columns** — no email, no student number, no notes. Add a comment at the top saying why there is no column for a last name.

#### What you should get back

About twenty lines of SQL. Read it before you run it. You are looking for two things: that students has exactly the columns you asked for, and that on delete cascade is on the class reference — that is what makes deleting a class in June actually delete its students.

#### Run it

Supabase, SQL Editor, paste, Run. Then Table Editor, and look at them. Seeing the empty tables is the point of the task.

#### If it added columns you did not ask for

It will sometimes add email or full\_name anyway, because that is what most rosters look like. **Delete those lines yourself before running it.** This is the first place the habit from task 1 earns its keep.

#### Task 3 ends when

Two tables exist and you have looked at them.

## The line that makes the wrong thing impossible

Everything else in this build is plumbing. This is the product.

### PROMPT 2 · THE CONSTRAINT

Add a CHECK constraint to the students table called **display\_name\_is\_shortened**. It must accept a single first name like **Jo** or **O'Neal**, or a first name followed by a single capital letter and a period like **Anthony S.**, optionally followed by a number like **Anthony S. 2**. It must reject anything containing a full surname, including **Anthony Spinozzi**, **Spinozzi**, **Anthony** and **Anthony S. Spinozzi**. Also cap the length at 24 characters. Give me the ALTER TABLE statement and explain the pattern in plain English, one clause at a time.

### What it looks like when it is right

```
constraint display_name_is_shortened check (
  display_name ~ '^[A-Za-z][A-Za-z'']*( [A-Z]\.( [0-9]+)?)?$'
  and length(display_name) <= 24
)
```

The doubled quote inside the pattern is how SQL writes a single apostrophe, and it is what lets O'Neal through. If yours has one quote there, the statement will not run.

### Now try to break it, in the SQL editor, before you build anything else

```
insert into students (class_id, display_name)
values ('00000000-0000-0000-0000-000000000000', 'Anthony Spinozzi');
```

You want the error. **new row for relation "students" violates check constraint "display\_name\_is\_shortened"**. If it stores instead, the constraint did not apply — check you ran the ALTER on the right table.

**That error message is the thing you are building.** Everything from here is a nicer way to avoid ever seeing it.

### Task 4 ends when

You have made the database say no to you, on purpose.

## TASK 5

### A page that talks to it

One HTML file. It asks for your URL and key, then reads the classes table.

#### PROMPT 3 · THE PAGE

Build a single HTML file, no frameworks and no build step, that runs by opening it in a browser. It has two boxes at the top for a Supabase project URL and an anon key, and a Connect button. It saves both to localStorage so I do not retype them, and it keeps working if localStorage is unavailable. On connect it reads the **classes** table through the Supabase REST endpoint using plain fetch — **not** the supabase-js library, since I am not installing anything — and lists the classes in a dropdown. Send the key in both the apikey and Authorization headers. If the request fails, show me the actual error message on the page rather than logging it to the console. Big text, big buttons.

#### The sentence that saves you twenty minutes

"not the supabase-js library, since I am not installing anything." Left to itself the AI writes `import { createClient }`, which needs npm, which needs a terminal, which is the cliff most teachers fall off. Plain fetch against the REST endpoint does the same job from a file you double-click.

#### Make one class so there is something to see

Easiest in the Supabase Table Editor: insert a row into classes with a name. Then reload your page.

#### Nothing appears and there is no error

Almost always Row Level Security. Supabase turns it on by default, and with no policy the anon key can read nothing — which looks exactly like an empty table. Prompt 4 on the prompts page fixes it, and the data page explains what you are turning on.

#### Task 5 ends when

Your class name appears in the dropdown, having come out of a database.

## TASK 6

### Shorten in the browser, before it sends

Layer 1. The one the student's family would care about most.

#### PROMPT 4 • THE SHORTENING

Add a function `shortenName` that turns a full name into a first name and a last initial. **Anthony Spinozzi** becomes **Anthony S.** It must also handle **Spinozzi, Anthony** written last name first, and give the same answer either way round. A single name like **Jo** is left alone, not mangled. A multi-part surname like **de la Cruz** or **Van Der Berg** surrenders only one letter. Then add a box I can type names into, showing a live preview of what each one becomes, so I can see the shortening happen before anything is sent.

#### Test it with the awkward ones yourself

| You type           | It should show |
|--------------------|----------------|
| Anthony Spinozzi   | Anthony S.     |
| Spinozzi, Anthony  | Anthony S.     |
| Maria de la Cruz   | Maria C.       |
| James Van Der Berg | James B.       |
| Jo                 | Jo             |
| jordan LEE         | Jordan L.      |

#### The two it gets wrong first time

**Last-name-first** rosters, which is how most school systems export them — without it, Spinozzi, Anthony becomes Spinozzi A. and you have stored the surname. And **mononyms**: a student with one name gets turned into Jo J. if nobody thought about it. Both are in the prompt for that reason.

**Why the live preview matters more than it looks.** It is the only moment a teacher actually sees the privacy decision happen. Without it this is a claim in a README; with it, it is a thing you watched work.

#### Task 6 ends when

You type a full name and watch it become a short one, before anything has been sent anywhere.

## TASK 7

### Paste a whole roster at once

Nobody types thirty names one at a time twice.

#### PROMPT 5 · THE PASTE

Add a big text box where I paste a whole class list, one name per line, straight out of whatever my school system exports. Shorten every line in the browser, show the preview as a list of **what I typed** to **what will be stored**, and add an **Add to class** button that inserts them all in one request with their sort order. Ignore blank lines. After a successful add, clear the box and refresh the list. If the insert fails, say so and **tell me nothing was added**, because the whole request is one statement.

#### Why one request and not thirty

A single insert either takes the whole class or none of it. That is much easier to reason about than a run that half worked and left you guessing which fourteen students made it. The phrase "tell me nothing was added" in the prompt is what gets you an honest message instead of a cheerful one.

#### Try it with your real roster

Export your actual class list and paste it. This is the first moment the tool is useful rather than interesting, and it is also the moment you find out what your school's export format really looks like.

#### Task 7 ends when

A real class is in the database, in one click, with no surname anywhere in it.

## TASK 8

### Two students with the same short name

You have an Anthony Spinozzi and an Anthony Sanders. Both shorten to Anthony S. This is not an edge case; it happens in most classes of thirty.

#### PROMPT 6 · COLLISIONS

Two students can shorten to the same name. When that happens, number them: the second **Anthony S.** becomes **Anthony S. 2**, the third becomes **Anthony S. 3**. It has to check against students **already in the class**, not only the ones in this paste, so adding a second Anthony next week still works. Show the numbering in the preview so I can see which is which before I commit.

#### The bug this prompt exists to prevent

Left alone, the code checks only within the batch you just pasted. Add Anthony Sanders in September and Anthony Spinozzi in October, and the October insert fails on the unique constraint with a message that will mean nothing to you at 7am. **"Already in the class, not only in this paste"** is the whole fix.

#### And now the awkward human bit

Somebody is Anthony S. 2 and they will notice. Tell them why on the first day: the tool does not keep last names, so it counts instead. Every student I have explained that to has thought it was reasonable. It is a better conversation than the one where a system holds their whole name for no reason.

#### Task 8 ends when

Two students who shorten the same way are both on the roster, and you can tell them apart.

## TASK 9

# Prove it to yourself

A button whose only job is to fail. It is the most valuable thing on the page.

### PROMPT 7 · THE PROOF

Add a panel called **Prove it to yourself** with a text box defaulting to **Anthony Spinozzi** and a button. The button sends that name to the database **bypassing the shortening**, and shows what comes back. If the database refuses it, say so clearly and show the actual error. If it succeeds, say loudly that this is wrong and the check constraint is missing. Make the failure case the one that looks reassuring.

### What you should see

```
Refused by the database, which is correct — new row for  
relation "students" violates check constraint  
"display_name_is_shortened"
```

### Why a button and not a note in the documentation

Because a claim you can press is different from a claim you can read. You will press it once a year when you have forgotten how any of this works, and it will tell you in two seconds whether the thing still holds. So will a principal, if you ever have to show them.

### If it stores the name

Stop. The constraint from task 4 did not apply — usually the ALTER ran against a different table, or ran before the table existed. Re-run it, delete the bad row, and press the button again.

### Task 9 ends when

You have triggered the refusal on purpose from inside your own tool.

## TASK 10

### Print it

The roster that matters most is the one on a clipboard when the network is down.

#### PROMPT 8 · PRINTING

Add a print stylesheet. When I print, hide the connection boxes, the paste area, the proof panel and every button — I want the class name and the numbered list of students and nothing else. Black on white, no background colors, and it must fit a page without me changing any print settings.

#### Test it properly, which means actually printing

Print preview is not the test. Send one to the machine in your building and look at it. This is where you discover that your roster of thirty-one runs onto a second page with one name on it.

#### The detail worth asking for

Numbered rows. A printed roster with numbers is countable at a glance, which is the entire reason you carry one.

#### Task 10 ends when

A piece of paper exists that you would actually use.

## TASK 11

### The de-identified copy

The version you can paste into an AI chat without thinking about it.

#### PROMPT 9 · THE EXPORT

Add a button that copies the class to the clipboard as **S1, S2, S3** in roster order, with no names at all. If the clipboard is blocked, show the text on the page so I can copy it by hand.

#### What this is actually for

Group arrangements, seating experiments, rotation schedules, anything where you want a second opinion from an AI about the *shape* of your class rather than about the people in it. You keep the S1-to-name mapping in your head or on paper. The chat never sees a child.

#### A first name is still a name

It is tempting to think Anthony S. is de-identified enough to paste anywhere. In a building where you teach, a first name and an initial identifies a specific child to anyone who works there. That is why this button exists and why it produces S1 rather than a shorter name.

#### Task 11 ends when

Your clipboard holds a class with nobody in it.

## TASK 12

### What happens in June

The task everybody skips, which is why so much student data outlives the student.

#### PROMPT 10 · THE ENDING

Add an archive control for a class, and a delete that really deletes — the class and every student in it. Make delete ask me to confirm by typing the class name, because it cannot be undone. Then tell me in one sentence what happens to the student rows when a class is deleted, based on my actual schema.

#### The answer you want back

"The students are deleted too, because `class_id` references `classes` with `on delete cascade`." If the AI cannot tell you that from your own schema, ask it to look again — and if the cascade is genuinely missing, add it now rather than in June.

#### Archive and delete are different promises

**Archive** means out of my way. **Delete** means gone. Tools that only archive let data quietly accumulate for years, and a list of every child you have taught since 2026 is a thing you never decided to keep.

#### Put a date on it

Pick a day — the last in-service day works — and delete last year's classes on it. A retention habit is worth more than a retention policy, and this one takes four minutes a year.

#### Task 12 ends when

You have deleted a test class and watched its students go with it.

## The six failures you will actually hit

Not generic advice. These are the six that happen, in the order they happen.

| What you see                                       | What it is, and the sentence that fixes it                                                                                                                                    |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| An empty list and no error at all                  | Row Level Security is on with no policy, so the anon key can read nothing — which looks identical to an empty table. Run the policy statements from <code>schema.sql</code> . |
| import statement is not defined                    | You got <code>supabase-js</code> code. Say: <b>rewrite this with plain fetch against the REST endpoint, no libraries and no build step.</b>                                   |
| violates check constraint when adding a real class | A name got past the shortening — usually a middle name or a suffix. Say: <b>show me which line of my paste produced that value.</b>                                           |
| duplicate key value violates unique constraint     | Two students shortened the same way and the numbering only checked within the batch. Task 8's prompt, with the "already in the class" clause.                                 |
| Failed to fetch, nothing else                      | Almost always a typo in the project URL, or a trailing slash. Paste it fresh from the Supabase settings page.                                                                 |
| It worked yesterday and today the page is blank    | Browser data was cleared, so the saved URL and key are gone. Paste them again. Your roster is untouched — that is the whole point of having a database.                       |

**The habit worth more than this table:** paste the error text into the chat *with what you were doing when it happened*. "It broke" gets you guesses. "I clicked Add to class with thirty names pasted and got this" gets you the answer.

## Every prompt, in one place – 1 to 5

Copy and paste. Square brackets mean put your own words in.

### 1 • THE TABLES

Write me a Postgres schema for a class roster, as SQL I can paste into the Supabase SQL editor. Two tables. classes: id, name, optional period, archived flag, created\_at. students: id, class\_id referencing classes with on delete cascade, display\_name, sort\_order integer, created\_at. Use uuid primary keys with gen\_random\_uuid(). Make display\_name unique within a class. Do not add any other columns. Add a comment at the top saying why there is no column for a last name.

### 2 • THE CONSTRAINT

Add a CHECK constraint to students called display\_name\_is\_shortened. Accept a single first name like Jo or O'Neal, or a first name followed by a capital letter and a period like Anthony S., optionally followed by a number like Anthony S. 2. Reject anything containing a full surname, including Anthony Spinozzi, Spinozzi, Anthony and Anthony S. Spinozzi. Cap the length at 24 characters. Give me the ALTER TABLE statement and explain the pattern one clause at a time.

### 3 • THE PAGE

Build a single HTML file, no frameworks and no build step, that runs by opening it in a browser. Two boxes for a Supabase project URL and anon key, and a Connect button. Save both to localStorage and keep working if localStorage is unavailable. On connect, read the classes table through the Supabase REST endpoint using plain fetch, not the supabase-js library, since I am not installing anything. Send the key in both the apikey and Authorization headers. Show the actual error on the page if the request fails. Big text, big buttons.

### 4 • LET THE PAGE READ AND WRITE

Give me the SQL to enable Row Level Security on both tables and add a policy that lets the anon key select, insert, update and delete on each. Then tell me in plain English who can reach my data once that policy exists.

### 5 • THE SHORTENING

Add a function shortenName turning a full name into a first name and last initial. Anthony Spinozzi becomes Anthony S. Handle Spinozzi, Anthony written last name first and give the same answer either way round. Leave a single name like Jo alone. A multi-part surname like de la Cruz or Van Der Berg surrenders only one letter. Add a box I can type into showing a live preview of what each name becomes before anything is sent.

## Every prompt, in one place – 6 to 12

### 6 · THE PASTE

Add a big text box where I paste a whole class list, one name per line. Shorten every line in the browser, preview what I typed against what will be stored, and add an Add to class button that inserts them all in one request with sort order. Ignore blank lines. Clear the box after a successful add. If the insert fails, tell me nothing was added, because the whole request is one statement.

### 7 · COLLISIONS

Two students can shorten to the same name. Number them: the second Anthony S. becomes Anthony S. 2. Check against students already in the class, not only the ones in this paste. Show the numbering in the preview.

### 8 · THE PROOF

Add a panel called Prove it to yourself with a text box defaulting to Anthony Spinozzi and a button that sends it to the database bypassing the shortening. If the database refuses it, say so and show the actual error. If it succeeds, say loudly that the check constraint is missing.

### 9 · PRINTING

Add a print stylesheet. When I print, hide the connection boxes, the paste area, the proof panel and every button. Class name and a numbered list of students, black on white, fitting the page with no print settings changed.

### 10 · THE EXPORT

Add a button copying the class to the clipboard as S1, S2, S3 in roster order with no names. If the clipboard is blocked, show the text on the page instead.

### 11 · THE ENDING

Add an archive control for a class, and a delete that removes the class and every student in it, confirmed by typing the class name. Then tell me what happens to the student rows on delete, based on my actual schema.

### 12 · WHEN THREE ROUNDS HAVE NOT FIXED IT

[Start a fresh chat.] Paste prompt 3 again, then add: also make sure that [the thing that kept going wrong].

## What is in this download, and what to do with it

### **roster.html**

The finished roster, built by exactly the twelve tasks in this pack and tested in a browser before publication. **Open it and read it** before you build your own — knowing what the target looks like makes every prompt land better. It works as-is: put your own URL and key in and it runs.

### **schema.sql**

The whole database, commented. Paste it into the Supabase SQL Editor and press Run, once. If you would rather have the working thing today and understand it this weekend, this file plus `roster.html` gets you there in about ten minutes.

### **Two ways to use this pack, both fine**

**Build it** — work the twelve tasks, prompt by prompt, and end up with something you understand well enough to change. **Run it** — use the two files as they are, and read the tasks when something confuses you. The first teaches more. The second gets a roster printed before homeroom.

**Keep both files somewhere you control.** Your own Drive, your own Documents. Not a shared staff folder, and not anywhere that publishes to the web — the next page explains exactly why.

## The key in your file, and who can read your roster

This page exists because every other tutorial about this leaves it out.

### The honest sentence

**Anyone who has both your project URL and your anon key can read and change this roster.** The policy you added in prompt 4 is what lets your page work, and it does not know the difference between your page and somebody else's.

### So three things follow, and they are not optional

**One.** The file lives on your computer. Never on a website, never in a public folder, never pasted into a forum when you are asking for help — strip the key first.

**Two.** Nothing sensitive goes in this database. Look at what is actually in there: first names, last initials, and the order they sit in.

**Three.** If you ever think the key has got out, Supabase can issue a new one in a few seconds. Do that, then paste the new one into your file.

### Now read those two sentences together, because this is the whole argument of the pack.

A leak is possible. What a leak would expose is a list that says "Anthony S." and "Maria C." **You did not make the breach impossible. You made it boring.** That is a far more reliable form of safety than being careful, because it keeps working on the day you are not.

### What this is not

Not a system of record, not a gradebook, and not a replacement for whatever your district runs. It is a roster you own, for your own classroom, made out of information you already carry around in your head.

## The three tiers, for this build specifically

The tier is decided by **where the data lives and who can reach it**, not by whether names appear.

### Tier 1 – build freely

The class name, the period, the structure of the tool, the schema, the code. All yours. None of it is about a child.

### Tier 2 – where this roster lives

First names and last initials, in a database only you have the key to, on a page that never leaves your machine. This is the tier the whole build is designed to sit in, and the constraint in task 4 is what keeps it there instead of drifting up a tier the first time somebody pastes a full roster into the wrong box.

### Tier 3 – stop, and ask the district

Anything traceable to a particular child beyond a name: grades, behavior records, IEP status, attendance, contact details. Also **any of this once it is shared** — **a staff folder, an emailed export, a chatbot window, a second teacher's account**. The data did not change. Who can reach it did, and that is what moves the tier.

**The part people get backwards.** Adding a database did not automatically push this build into tier 3. Sharing it would. A tool on a server that only you can reach is closer to a notebook in your desk than to a district system, and the difference is who holds the key.

### Not legal advice

Your district's policy, your state's law, FERPA and COPPA are the authorities. This is a way of thinking that keeps you well inside them, not a substitute for asking.

## Eight things to try before you trust it

Ten minutes. Do these with a fake class, not your real one.

| Try this                              | You want                                  |
|---------------------------------------|-------------------------------------------|
| Paste Spinozzi, Anthony               | Anthony S., not Spinozzi A.               |
| Paste a student with one name         | Left alone, not Jo J.                     |
| Add two students who shorten the same | Both there, second numbered               |
| Close the tab and reopen it           | The roster is still there                 |
| Open the file on a different computer | Same roster, after pasting the key        |
| Press the proof button                | A refusal, quoting the constraint by name |
| Print it                              | Names and numbers, no buttons, one page   |
| Delete the fake class                 | Its students gone too                     |

### The one that catches the most

**The second computer.** It is the only test that proves you built what you think you built — a tool whose data is somewhere other than the machine in front of you. If the second computer shows an empty roster, something is saving locally that should not be.

### And one you cannot test by looking

Whether a surname ever reached the server. Open your browser's network tab while you add a class and read what was actually sent. It should say Anthony S. If it says anything longer, the shortening is running in the wrong place.

## Postgres is blunt, and that is a kindness

Database errors look frightening and are the most useful text in this whole build.

```
new row for relation "students" violates check  
constraint "display_name_is_shortened"
```

### Three pieces, left to right

**new row for relation "students"** — where it happened. Not classes, not somewhere mysterious. The students table.

**violates check constraint** — what kind of rule. A CHECK is a rule about the contents of one row, which means the problem is in the value, not in your connection or your key.

**"display\_name\_is\_shortened"** — which rule, by the name you gave it in task 4. This is why naming constraints is worth the two extra words.

### The other one you will meet

```
duplicate key value violates unique constraint  
"students_class_id_display_name_key"
```

Same shape. The name tells you the columns involved: `class_id` and `display_name`. Two students in one class shortened the same way, and the numbering did not catch it.

**The rule for getting help fast:** paste the whole error, not a summary of it. The constraint name is the single most useful word in the sentence and it is the first thing people paraphrase away.

## What to say to thirty students

You built a tool that treats them carefully. Tell them.

### Sixty seconds, roughly like this

"This is the roster I use. It has your first name and the first letter of your last name, and that is all it has — not your ID, not your grades, not your birthday. It is set up so it *cannot* keep your last name, not just so I remember not to type it. If two of you come out the same it will number you, and that is why one of you is Anthony S. 2."

### Why bother

Because students are told constantly that software watches them, and they are mostly right. Showing them one piece that was deliberately built to know less is a better lesson about technology than any unit on digital citizenship, and it takes a minute.

### If somebody asks a hard question

"Could someone else see it?" is a fair question and the answer is yes, in principle, if they had my key — and what they would see is a list of first names. **Answer it straight.** A student who catches you overselling the safety of a system learns the wrong lesson very efficiently.

### The ask worth making

Ask them what else the school stores about them, and who they think can see it. You will get a better discussion out of that than out of any prepared question.

## When one file is enough, and when it is not

Honest answer: most classroom tools should be one file. This is not most tools.

### Stay with one file when

Only you use it · only on one machine · losing the data would be annoying rather than serious · it is a timer, a picker, a generator, a checklist · you want it working in four minutes.

### You have outgrown one file when

The same information has to be in front of two machines · losing it would cost you a real afternoon · something else you build needs to read the same list · it has to survive a browser being cleared by somebody else's IT policy.

### The trap in between

**Do not put a database behind something that did not need one.** You will have taken on a key to protect, a service to remember, and a second thing that can be down, in exchange for nothing. The question is never "is a database better" — it is "what does this tool do that one file cannot."

**The roster is the honest first case,** because a class list is the thing every other classroom tool wants to read. Once it lives somewhere with a key, the next tool you build can use it instead of asking you to type thirty names again.

## How this pack is built, and what it does not claim

### Where this pattern comes from

It is not theoretical. The three-layer approach — shorten in the browser, refuse in the database, no column to put it in — is taken from software I built and run for my own high school woodshop course, which has been in daily use with real students for a full school year and is still being extended. The roster in this pack is that design, reduced to the smallest thing that teaches it.

### What was actually tested before this went out

The constraint pattern was run against accepting and rejecting inputs — Jo, O'Neal, Anthony S., Anthony S. 2 accepted; Anthony Spinozzi, Spinozzi, Anthony, Maria de la Cruz, Anthony S. Spinozzi refused. The included roster.html was driven in a real browser against a stand-in database enforcing that same constraint: connect, create a class, paste five names including a deliberate collision, confirm the shortened forms are what reach the database, trigger the refusal on purpose, remove a student, reload, and handle an unreachable database. No unexpected console errors.

### What is NOT claimed

- **Not tested against a live Supabase project** — the browser test used a stand-in that enforces the same rules. Your first real connection is yours to make, and task 5 tells you what a good one looks like.
- **Not a secure multi-user system.** Anyone with your URL and key can read the roster. That is stated plainly rather than hedged.
- **Not a system of record,** not a gradebook, not an SIS.
- **Not legal advice.** Your district's policy, your state's law, FERPA and COPPA are the authorities.

### Why the AI is told what not to do so often

Most of the prompts in this pack spend a clause forbidding something — no extra columns, no libraries, no build step. That is not fussiness. Left to itself a model builds what most software looks like, and most software collects more than it needs. Saying no is the part of prompting that nobody teaches.

## ONE LAST THING

### If this was useful

#### **Leaving a review earns you TPT credit**

TPT gives you credit toward future purchases for every review you leave, and reviews are how other teachers find work like this. It takes about thirty seconds.

#### **If something did not work, tell me before you rate it**

The Q&A on the listing reaches me directly. A database error you cannot get past is almost always one line, and I would far rather fix it than have you stuck with a file that does not run.

#### **The one thing to take away if you take nothing else**

Decide what your tool must never know, then make it structurally unable to know it. Not a rule you follow. A shape it has. Everything else in this pack is one worked example of that idea.

#### **Built and audited by two certified teachers.**

Every prompt here was run. The constraint was tested against names designed to break it. The app was driven in a browser, including the failures.